

# Porównanie efektywności metod stochastycznego gradientu i metod globalnych w zadaniach optymalizacji globalnej

Michał Brzeziński

Wydział Elektroniki i Technik Informacyjnych

## 1 Streszczenie

Celem pracy było porównanie algorytmów stochastycznych z algorytmami globalnymi, oraz określenie zakresu kompetencji każdego z nich za pomocą prostej sieci neuronowej, i benchmarku CEC2013

## 2 Wstęp

### Cel i zakres pracy

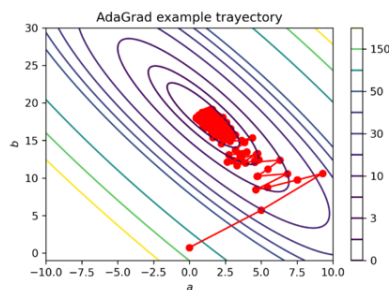
Praca ma na celu porównanie algorytmów stochastycznych i globalnych. Określić mocne strony każdego algorytmu, oraz wskazanie kierunków na potencjalny rozwój

## 3 Algorytmy

### Gradientowe

#### AdaGrad

Algorytm AdaGrad (Adaptive Gradient Algorithm) jest zmodyfikowanym algorytmem stochastycznym spadku gradientu, dla którego współczynnik uczenia jest ciągle aktualizowany na podstawie historii gradientów poprzednich kroków. Dynamicznie zmieniający się współczynnik uczenia pozwala płynniej przechodzić z fazy eksploracji w fazę eksploatacji.



Dla  $g_t = \nabla Q_i(w)$

$$G = \sum_{\tau=1}^t g_{\tau} g_{\tau}^T$$

$$w := w - \eta \operatorname{diag}(G)^{-1/2} \odot g_t$$

$w$  - aktualny wektor pozycji

$\eta$  - współczynnik uczenia

$Q_i(w)$  - funkcja celu w punkcie ' $w$ ' i dla zbioru danych treningowych ' $i$ '

$g_t$  - średnia gradientów funkcji celu w momencie ' $t$ ' dla wszystkich próbek w batchu ' $i$ '

$G$  - historia złożona z sumy kwadratów wektorów poprzednich iteracji

### Adam

Algorytm Adam (Adaptive Moment Estimation) wprowadza koncept adaptacji współczynnika uczenia za pomocą momentum oraz RMSProp do standardowego algorytmu stochastycznego spadku gradientu. Adaptuje krok uczenia w każdym wymiarze oddzielnie, łącząc momentum i skalowanie drugiej chwili gradientu. Dynamiczny współczynnik uczenia algorytmu łączy dwa elementy - momentum obliczane na podstawie średniej ważonej gradientu z poprzednich kroków, oraz średnia ważona kwadratów gradientów (druga chwila gradientu), której zadaniem jest optymalizowanie wartości wzmacniając małe gradienty i wygłaszając za duże wartości. Można porównywać to rozwiązanie do kontrolera PID w automatyce. Momentum określa kierunek, druga chwila gradientu stabilizuje ruch algorytmu

dla :

$$m_w^{(t)} := \beta_1 m_w^{(t-1)} + (1 - \beta_1) \nabla Q(w)^{(t)}$$

$$\hat{m}_w^{(t)} = \frac{m_w^{(t)}}{1 - \beta_1^t}$$

$$v_w^{(t)} := \beta_2 v_w^{(t-1)} + (1 - \beta_2) (\nabla Q(w)^{(t)})^2$$

$$\hat{v}_w^{(t)} = \frac{v_w^{(t)}}{1 - \beta_2^t}$$

$$w^{(t)} := w^{(t-1)} - \eta \frac{\hat{m}_w^{(t)}}{\sqrt{\hat{v}_w^{(t)} + \epsilon}}$$

Gdzie:

$\eta$  - współczynnik uczenia

$Q(w)$  - funkcja celu dla wektora ' $w$ '

$\epsilon$  - niewielka liczba zapobiegająca dzieleniu przez zero

$\hat{m}_w^{(t)}$  - momentum wektora ' $w$ ' podczas kroku ' $t$ '

$\beta_1$  - współczynnik zapomnienia momentum

$\hat{v}_w^{(t)}$  - druga chwila gradientu wektora ' $w$ ' podczas kroku ' $t$ '

$\beta_2$  - współczynnik zapomnienia drugiej chwili gradientu

**L-BFGS-B**

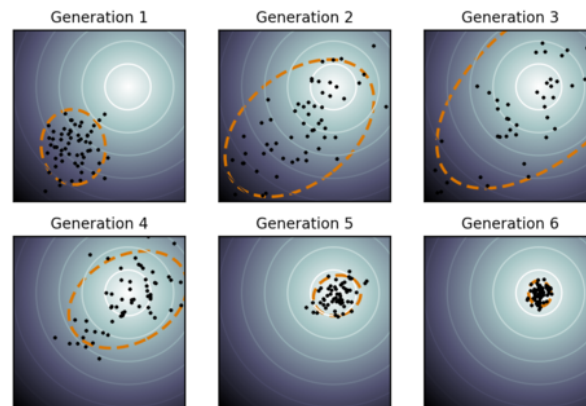
Algorytm BFGS wykorzystuje aproksymacje hesjanu funkcji, pozwalając na skalowanie kroku mając wgląd nie tylko w przebieg funkcji, ale też jej gradientu. Pozwala na dokładniejsze wybory kierunku i wielkości kroków.

L-BFGS-B to rozszerzenie algorytmu BFGS o ograniczenia przestrzeni (Box constraints) oraz pamięci (Limited memory). Przestrzeń rozwiązań jest ograniczona, a hesjan jest aproksymowany na podstawie macierzy ustalonej liczby ostatnich kroków, pracuje na ograniczonej pamięci nie wykorzystując pełnej macierzy hesjanu.

**Globalne****CMAES**

Algorytm CMA-ES to algorytm ewolucyjny, który na podstawie populacji rodziców określa zakres przestrzeni na której generuje nowych potomków.

Tworzy populację kandydatów, ocenia ich jakość i na jej podstawie aktualizuje średnią i macierz kowariancji rozkładu wielowymiarowego, którą wykorzystuje do losowania nowej generacji



Rysunek 1: przykładowy przebieg algorytmu w CMAES

**NL-SHADE-RSP-MID**

Algorytm NL-SHADE-RSP-MID to rozszerzenie algorytmu DE, adaptacyjnie ustawiające parametry krzyżowania i mutacji  $Cr$  i  $F$ , na podstawie populacji oraz archiwum rozwiązań.

NL-SHADE:

- Dla każdego osobnika generowany jest wektor mutant na podstawie trzech losowo wybranych osobników z populacji i potencjalnie archiwum.

- Do operatora różnicowego mutacji może zostać wykorzystany punkt wylosowany z archiwum, które jest złożone z rodziców, którzy zostali zastąpieni potomkami z lepszym wynikiem.
- Indywidualnie wartości Cr i F są generowane wykorzystując losową wartość z pamięci Cr i F jako centra rozkładów.
- Określamy dla każdego osobnika poprawę wartości funkcji celu względem poprzedniej generacji oraz wartości Cr i F użyte na nich. Na podstawie tych danych, średnią ważoną Lehmera obliczamy najlepszą preferowaną wartość Cr i F dla generacji. Wynik jest zapisywany na jednej z pozycji w pamięci Cr i F.
- Pod koniec generacji porównywane są wartości funkcji celu osobników powstałych z mutacji, które wykorzystały archiwum z tymi, które tego nie zrobiły i, na podstawie sukcesu potomków, obliczane jest prawdopodobieństwo użycia archiwum w mutacji dla następnej generacji.
- Rozmiar populacji oraz archiwum jest zmieniany w zależności od liczby wywołań funkcji celu w stosunku do limitu.

RSP:

- Jeśli punkt nie mieści się w podanej przestrzeni danych, jest generowany ponownie.

MID:

- Populacja jest klastrowana względem pozycji w przestrzeni rozwiązań, porównywane są wartości funkcji celu żeby wyznaczyć najlepszy podział, oraz porównać go z pełnym zbiorem żeby określić czy populacja ma zgrupowanie kilku punktów. Wyznaczany jest centroid najlepszego zgrupowania. Jeśli centroid się nie zmienia, oznacza to stagnację i algorytm jest zatrzymywany

## 4 Metody ewaluacji

### Metody testowania

Do porównania wyników każdego eksperymentu wykorzystano funkcję ECDF (Empirical Cumulative Distribution Function) oraz wykres średnich wartości w czasie z odchyleniem standardowym, oba wykresy w skali logarytmicznej. ECDF przedstawia skumulowany ułamek progów błędu osiągniętych przez każdy algorytm w kolejnych punktach w czasie. Ponieważ wszystkie algorytmy są oceniane względem tych samych progów, metoda dobrze wizualizuje, jak każdy z algorytmów radził sobie w różnych etapach eksperymentu. Metoda ta uniemożliwia spadek osiągniętych progów, co znacząco poprawia czytelność wykresów

### Benchmarki CEC

#### Opis problemu

Congress on Evolutionary Computation (CEC), czyli Kongres Optymalizacji Ewolucyjnej, to konferencja naukowa organizowana przez IEEE, której celem

jest publikacja standardowych funkcji testowych do oceny algorytmów optymalizacji.

Benchmarki CEC to zestaw standardowych testów optymalizacyjnych opracowanych przez Congress on Evolutionary Computation (CEC). Ich celem jest obiektywne porównanie algorytmów optymalizacyjnych.

### Implementacja

Do testów wykorzystano CEC2013, zbiór funkcji używanych na Kongresie Optymalizacji Ewolucyjnej w 2013 roku.

Jest to powszechnie stosowany punkt odniesienia do testowania algorytmów sztucznej inteligencji.

Zastosowano się do wytycznych ustalonych podczas konferencji. Każdy algorytm był uruchamiany 51 razy dla 10, 30 oraz 50 wymiarów, przy ograniczonej liczbie wywołań funkcji ewaluacyjnej.

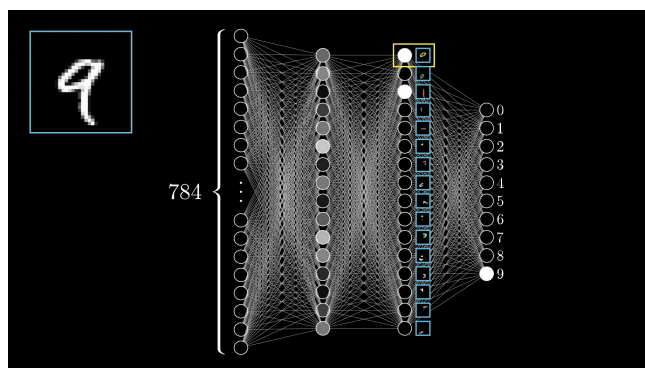
### Sieci neuronowe

#### Opis problemu

- Benchmarki CEC2013 są przystosowane do pracy na relatywnie niewielkiej liczbie wymiarów.
- Nie pozwala to na określenie pełnego zakresu możliwych problemów optymalizacyjnych.
- W sieciach neuronowych liczba wag liczona jest zwykle w skali tysięcy.

#### Implementacja

By porównać algorytmy stochastyczne i globalne zaimplementowana została prosta sieć neuronowa do klasyfikacji obrazów z bazy MNIST. Dla obu rodzajów algorytmów struktura sieci składa się z warstwy wejściowej z 784 neuronami, warstwy losowego embeddingu o wymiarze 200, dwóch warstw ukrytych po 16 neuronów każda oraz warstwy wyjściowej z 10 neuronami. Wielkości warstw ukrytych zostały wybrane na podstawie najczęściej spotykanych wartości w literaturze.

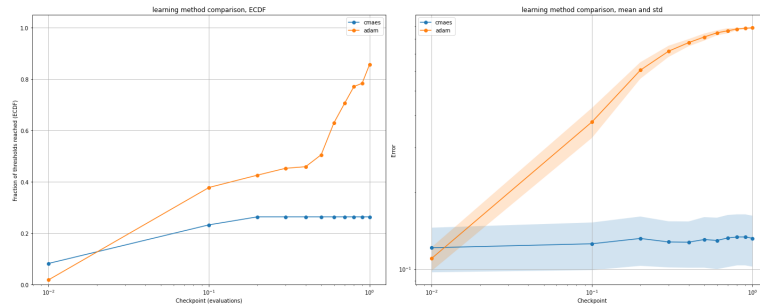


Rysunek 2: model przykładowej sieci neuronowej

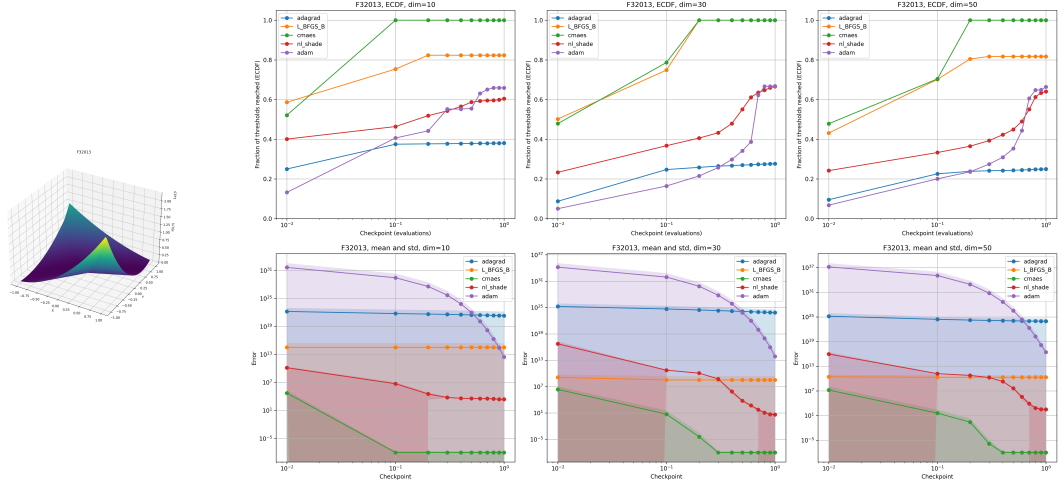
- Wykorzystując lemat Johnsona-Lindenstraussa, jesteśmy w stanie oszacować jak redukcja wymiarów przestrzeni wpływa na zachowanie zależności pomiędzy osobnikami w tej przestrzeni. Dzięki temu możliwe jest osadzenie danych w przestrzeni o mniejszej liczbie wymiarów kontrolując zniekształcenia zależności między osobnikami.
- dla danego  $\epsilon \in (0, 1)$  i zbioru  $X \subset R^{dim}$  o wielkości  $|X| = n$  istnieje mapowanie  $f : R^{dim} \rightarrow R^m$  gdzie nowy wymiar  $m = O(\frac{\log n}{\epsilon^2})$  które osadza zbiór  $X$  do przestrzeni  $R^m$  ze zniekształceniem maksymalnym równym  $\epsilon$ .
- Warstwa embeddingu losowego Dla 784 wymiarów mapuje osobniki do przestrzeni 200-wymiarowej więc można się spodziewać zniekształceń w zależnościach rzędu 0.12
- Jako reprezentant algorytmów stochastycznych został wykorzystany algorytm Adam, natomiast do algorytmów globalnych użyty został algorytm CMAES.
- W przypadku rozwiązania stochastycznego Zastosowano standardową metodę backpropagacji do uczenia i adaptacji wag i biasów. Optymalizowano funkcję kosztu.
- Dla algorytmów globalnych problem ma wymiarowość równą sumie wszystkich trenowanych parametrów, czyli sumę wag i biasów warstw ukrytych i wyjścia. Łącznie 3658 wymiarów.
- Optymalizowane było accuracy. Każdemu osobnikowi przydzielano ocenę na podstawie 20 losowo wybranych przykładów ze zbioru treningowego.
- Jako wspólną jednostkę czasu przyjęto jedno przejście przez sieć neuronową. W przypadku globalnego algorytmu CMA-ES oznaczało to inkrementację licznika po każdej ocenie pojedynczego osobnika, natomiast w przypadku stochastycznego algorytmu licznik był zwiększany po każdej parze operacji przejścia w przód(forward pass), oraz backpropagacji.
- Dla każdego algorytmu zapisywano najlepsze rozwiązania po przejściu przez daną część zasobów. Rozwiązania te były następnie oceniane wykorzystując zbiór testowy, aby określić zmianę accuracy w czasie oraz ECDF

## 5 Wyniki eksperymentów

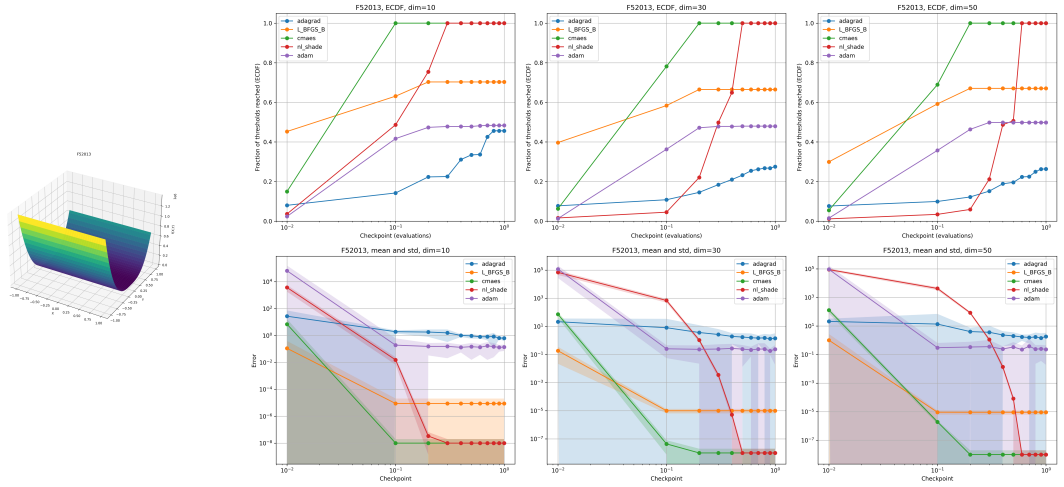
### wybrane wykresy



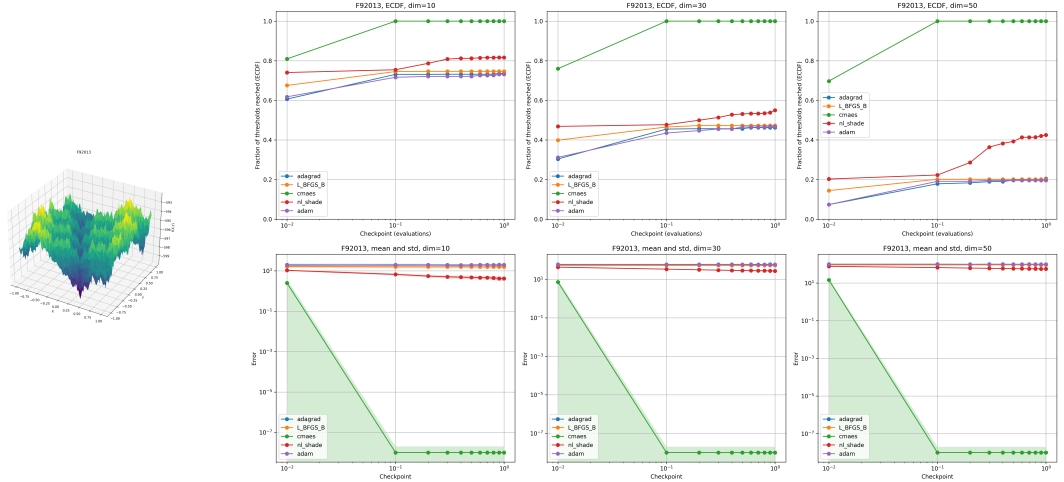
Rysunek 3: model przykładowej sieci neuronowej



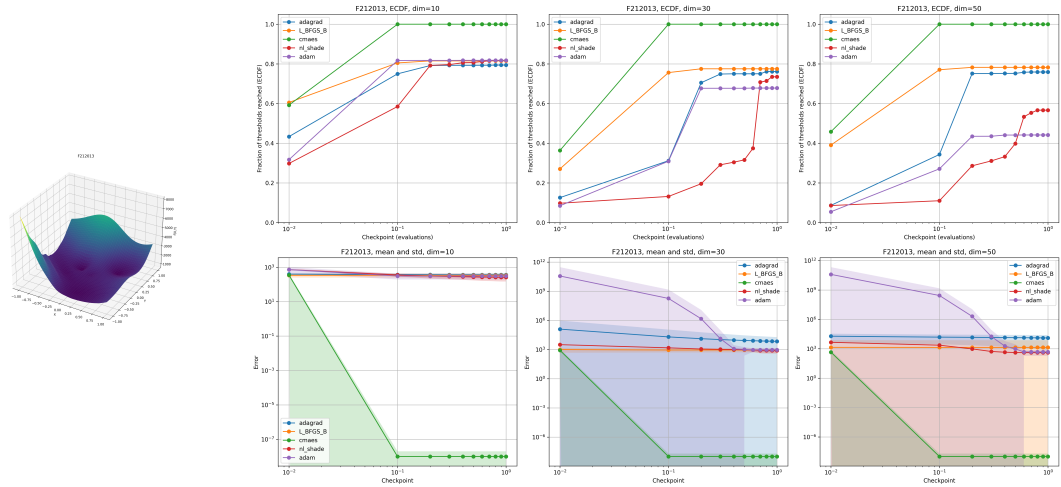
Rysunek 4: F3 - Rotated Bent Cigar Function



Rysunek 5: F5 - Different Powers Function



Rysunek 6: F9 - Rotated Weierstrass Function

Rysunek 7: F21 - Composition Function 1 ( $n=5$ , Rotated)

## Obserwacje

### Algorytmy globalne vs stochastyczne

- Algorytmy globalne osiągają wyraźną przewagę w zadaniach optymalizacyjnych CEC, zwłaszcza w niskich i średnich wymiarach, natomiast algorytmy stochastyczne sprawdzają się lepiej w optymalizacji sieci neuronowych.

### Adam i Adagrad

- Pomimo zastosowania momentum, oba algorytmy osiągają podobne wyniki.
- Nie działają optymalnie, gdy zbiory minimów lokalnych funkcji (doliny funkcji) tworzą „rynny” nierównoległe do osi układu współrzędnych. Można porównać funkcję F5 - “Different Powers Function” z “doliną” równoległą do osi, oraz F3 - “Rotated Bent Cigar Function”.
- Algorytm Adam, pomimo wielu zalet, nie zawsze radzi sobie lepiej względem algorytmu Adagrad. Widać problemy gdy funkcja ma gwałtowne spadki jak funkcja F21 - “Composition Function 1 ( $n=5$ , Rotated)”. Można zauważyć, że radzi sobie znacząco gorzej i różnica w wynikach jest większa im więcej jest wymiarów.

### L-BFGS-B

- Wykazuje podobne ograniczenia co Adam i Adagrad. Można zauważyć te same problemy dla funkcji o „rynnowej” strukturze minimów. Jednak zwykle osiąga lepsze wyniki, czasem porównywalne z algorytmami globalnymi.
- Dla większości przypadków krzywe ECDF tego algorytmu często przypominają wykresy Adagrad, przesunięte o około 20% w zakresie przebiegłych progów jakości.



- Jest bardzo wrażliwy na szum i chaotyczność funkcji, ponieważ aproksymacja hesjanu staje się niestabilna (F9 - “Rotated Weierstrass Function”).
- Dla funkcji “Rotated Lunacek Bi\_Rastrigin Function” widać, że czasami Adagrad i Adam potrafią przebić jakość rozwiązań algorytmu L-BFGS-B, jeśli funkcja ma wystarczająco dużo szumu i jest wystarczająco chaotyczna wizualnie.

## CMA-ES

- Osiąga najlepsze wyniki w funkcjach CEC2013 w porównaniu do pozostałych algorytmów.
- Dzięki rozproszeniu populacji i estymacji średnich wartości parametrów, pozwala na dokładniejszą eksplorację krajobrazu funkcji i szybsze znalezienie ekstremów przy mniejszej liczbie wymiarów.
- Jak teoretyzowano tworząc nl-shade-rsp-mid, prawdopodobnie ma to związek z teorią, że środek populacji ma większe szanse być lepszą aproksymacją ekstremum od pojedynczego punktu.
- Czas obliczeń algorytmów stochastycznych rośnie liniowo względem ilości wymiarów, natomiast algorytmów globalnych potęgowo. Z wyników CEC widać, że algorytmy globalne radzą sobie o wiele lepiej od stochastycznych dla większej liczby wymiarów, jednak dla problemu optymalizacji sieci neuronowej algorytmy globalne przestają być opłacalne.

## nl-shade-rsp-mid

- Działa wyraźnie gorzej niż CMA-ES jako algorytm globalny. Optymalizacja zajmuje znacząco dłużej, a punkty nie są równie dobrej jakości.
- Zmiana metody krzyżowania w połowie przebiegu sugeruje, że algorytm był zaprojektowany z nadzieją na dłuższy czas działania, w przeciwieństwie do CMA-ES, który konsekwentnie wykorzystuje tę samą strategię przez cały przebieg.

## Sieci neuronowe

- Klasyczne sieci z algorytmem stochastycznym minimalizującym funkcję straty nie zachowują zależności między neuronami — każda aktualizacja wagi jest lokalna, liniowa i niezależna.
- Algorytmy globalne zmieniają wszystkie wagi jednocześnie, co zwiększa złożoność obliczeniową i ogranicza skalowalność w zastosowaniach sieciowych.

## 6 Podsumowanie i wnioski

### Podsumowanie wniosków obu sposobów

#### Algorytmy stochastyczne

- Poruszają się niezależnie w każdym wymiarze, co sprawia, że są wrażliwe na kierunki przekątne w krajobrazie funkcji.

### **Wpływ szumu**

- Im bardziej złożony algorytm stochastyczny, tym bardziej wrażliwy jest na szum i chaotyczność funkcji.

### **Charakter algorytmów**

- Adam, Adagrad i L-BFGS-B aktualizują parametry jeden wymiar na raz, co powoduje problemy w funkcjach, których minima lokalne tworzą obiekt nierównoległy do osi układu (np. F3 - “Rotated Bent Cigar Function”).
- CMA-ES i inne algorytmy globalne, dzięki rozproszonej populacji, radzą sobie lepiej z kierunkami przekątnymi i wykrywaniem ekstremów w krajobrazie funkcji.

### **Skalowalność**

- Algorytmy stochastyczne skalują się liniowo względem liczby wymiarów.
- Algorytmy globalne, jak CMA-ES, skalują się potęgowo, co ogranicza ich opłacalność przy dużych wymiarach, mimo ich dokładności dla mniejszych wymiarów.

### **Cechy specyficzne algorytmów**

- Zmiana strategii w trakcie działania, np. w nl-shade-rsp-mid, sugeruje nastawienie na długotrwałą eksplorację, w odróżnieniu od CMA-ES, który utrzymuje stałą strategię.

### **Osiągnięte cele**

- Zaimplementowane środowisko do testowania algorytmów sztucznej inteligencji, z możliwością rozszerzenia o kolejne w przyszłości.
- Opisano działania algorytmów.
- Określone zostały zakresy kompetencji podanych algorytmów.
- Zostały porównane ich wady oraz zalety.

### **Kierunki dalszego rozwoju**

- Aktualne rozwiązania wykorzystujące algorytmy globalne do sieci neuronowej nie przenoszą informacji o zależności pomiędzy neuronami, wagi są spłaszczane do jednego wymiaru, ignorując powiązania pomiędzy neuronami. Potencjalnie możliwe jest zaprojektowanie algorytmu globalnego dostosowanego specyficznie pod naukę sieci neuronowych, który poradziłby sobie lepiej.
- Implementacja kolejnych algorytmów i porównanie ich z obecnymi wynikami.
- Rozszerzenie testów o kolejne benchmarki, takie jak CEC2017.
- Badanie hybrydowych metod optymalizacji, łączących algorytmy globalne z lokalnymi lub stochastycznymi.

- Opracowanie metod redukcji wymiarów dla zadań optymalizacji sieci neuronowych metodami globalnymi.
- Badanie wpływu szumu w danych wejściowych lub funkcji kosztu na skuteczność algorytmów.

# Bibliografia

## Literatura

1. Cornell University. AdaGrad. <https://optimization.cbe.cornell.edu/index.php?title=AdaGrad>. Accessed: 2026-01-27.
2. Wikipedia. Stochastic gradient descent – AdaGrad. [https://en.wikipedia.org/wiki/Stochastic\\_gradient\\_descent#AdaGrad](https://en.wikipedia.org/wiki/Stochastic_gradient_descent#AdaGrad). Accessed: 2026-01-27.
3. PyTorch. torch.optim.Adam — PyTorch documentation. <https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html>. Accessed: 2026-01-27.
4. Wikipedia. Stochastic gradient descent – Adam. [https://en.wikipedia.org/wiki/Stochastic\\_gradient\\_descent#Adam](https://en.wikipedia.org/wiki/Stochastic_gradient_descent#Adam). Accessed: 2026-01-27.
5. GeeksforGeeks. Deep Learning Adam Optimizer. <https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>. Accessed: 2026-01-27.
6. Wikipedia. Limited-memory BFGS. [https://en.wikipedia.org/wiki/Limited-memory\\_BFGS](https://en.wikipedia.org/wiki/Limited-memory_BFGS). Accessed: 2026-01-27.
7. Liu, D.C., Nocedal, J.: A Limited Memory Algorithm for Bound Constrained Optimization. <https://dl.acm.org/doi/pdf/10.1145/279232.279236>. Accessed: 2026-01-27.
8. E-sezam. Materiały dydaktyczne. <https://esezam.okno.pw.edu.pl/mod/book/view.php?id=1245&chapterid=1950>. Accessed: 2026-01-27.
9. Tanabe, R., Fukunaga, A.: NL-SHADE-RSP Algorithm with Adaptive Archive and Selective Pressure for CEC 2021 Numerical Optimization. [https://www.researchgate.net/publication/353782316\\_NL-SHADE-RSP\\_Algorithm\\_with\\_Adaptive\\_Archive\\_and\\_Selective\\_Pressure\\_for\\_CEC\\_2021\\_Numerical\\_Optimization](https://www.researchgate.net/publication/353782316_NL-SHADE-RSP_Algorithm_with_Adaptive_Archive_and_Selective_Pressure_for_CEC_2021_Numerical_Optimization). Accessed: 2026-01-27.